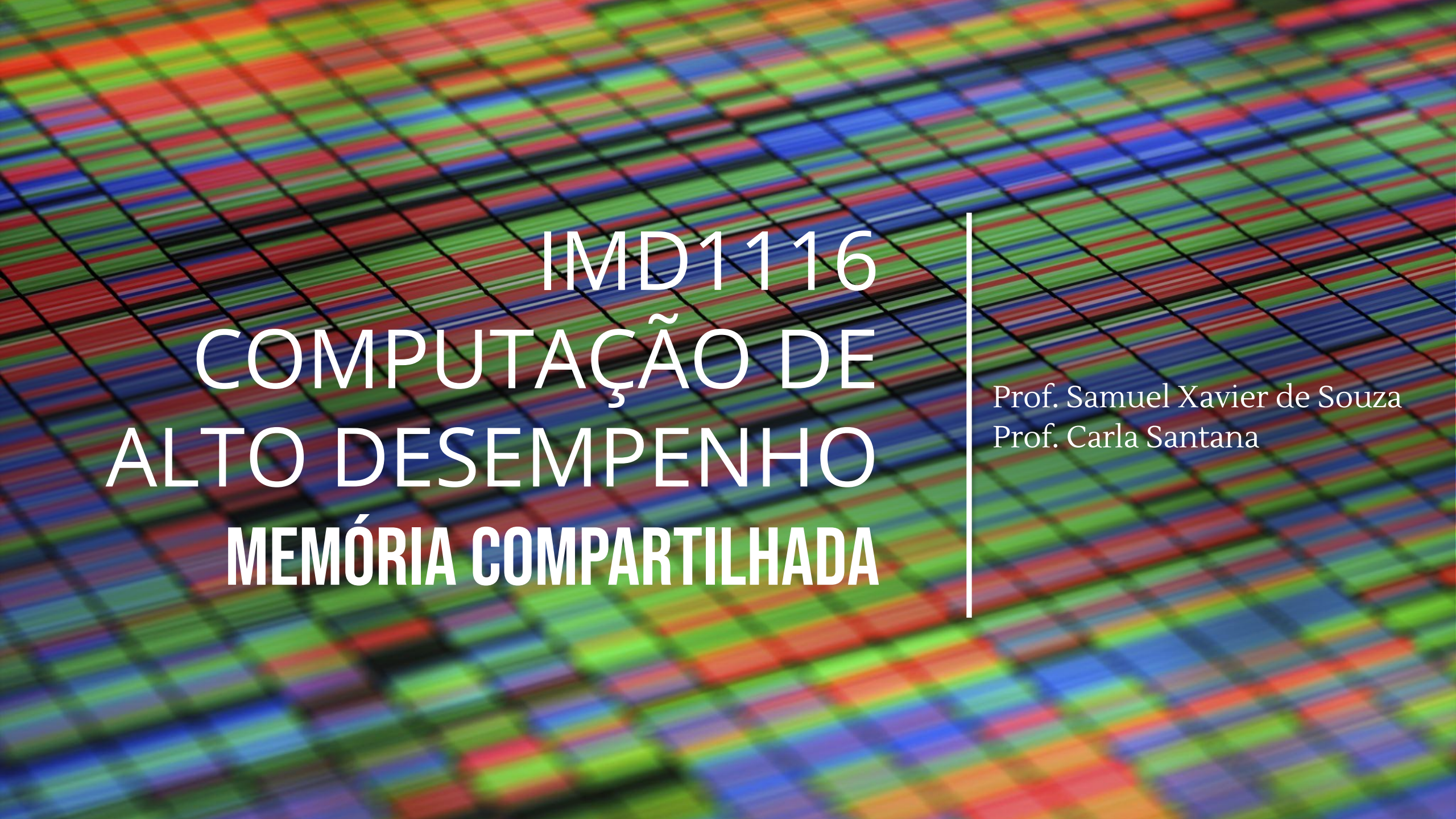




IMD1116

COMPUTAÇÃO DE

ALTO DESEMPENHO

MEMÓRIA COMPARTILHADA

Prof. Samuel Xavier de Souza
Prof. Carla Santana

PROGRAMAÇÃO EM MEMÓRIA COMPARTILHADA

- Objetivos deste tópico
 - Compreender o paradigma de memória compartilhada
 - Compreender os desafios de programar em memória compartilhada
 - Conhecer as ferramentas básicas de programação em memória compartilhada

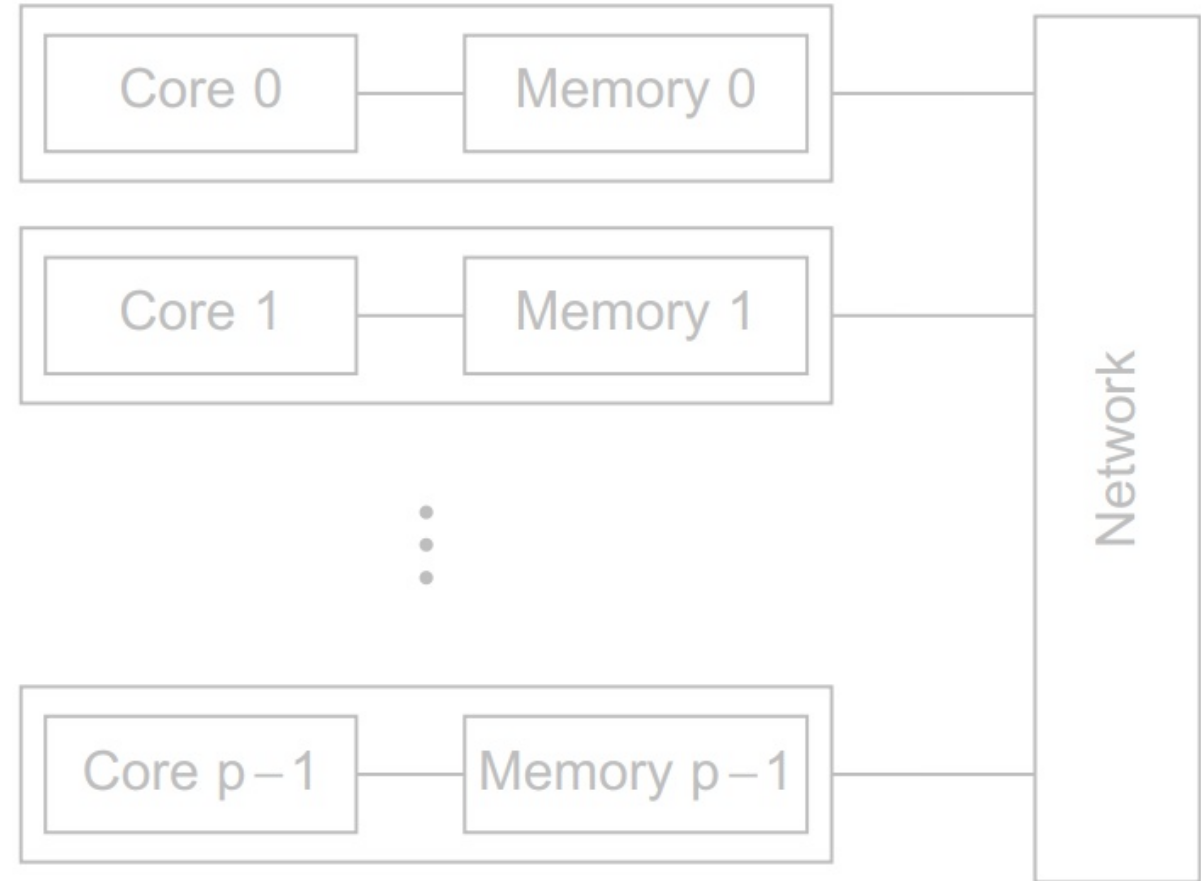
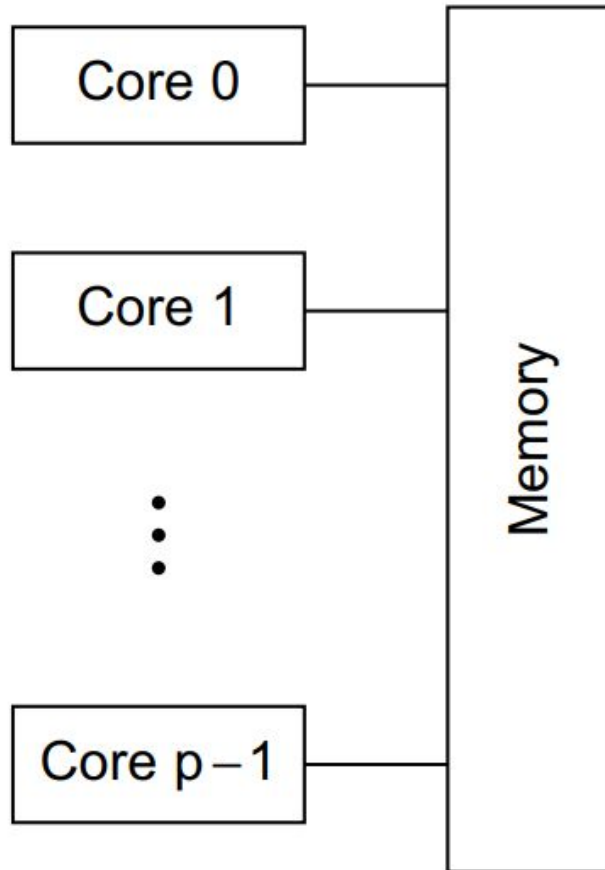
2ª Atividade extra da 1ª Unidade

Defina, de forma sucinta, os seguintes conceitos relacionados à concorrência e sistemas operacionais:

- Condição de corrida
- Região Crítica
- Exclusão mútua
- Deadlock
- Troca de contexto em Sistemas Operacionais

Você terá **15** minutos para elaborar e entregar a resposta por escrito. Após a entrega, a professora poderá sortear alunos para apresentar a resposta durante a aula.

MEMÓRIA COMPARTILHADA X MEMÓRIA DISTRIBUÍDA



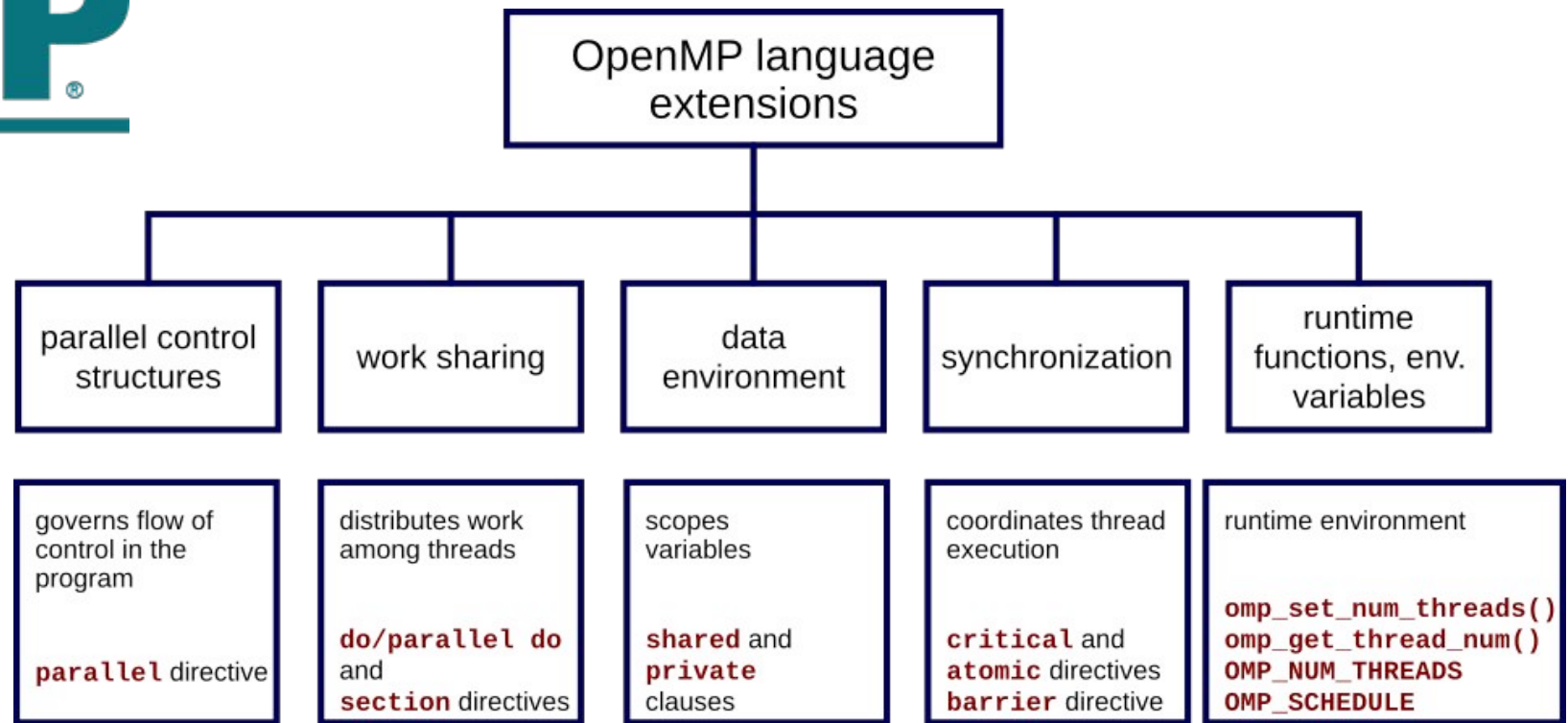
PROGRAMAÇÃO COM MEMÓRIA COMPARTILHADA

- Comunicação entre as tarefas é realizada por meio de variáveis **armazenadas em um espaço de memória compartilhado** .
- Processos X Threads

PROGRAMAÇÃO COM MEMÓRIA COMPARTILHADA

- Comunicação entre as tarefas é realizada por meio de variáveis **armazenadas em um espaço de memória compartilhado** .
- Processos X Threads
- Caso de processos, é necessário requisitar ao **sistema operacional** a definição de um espaço de memória compartilhado por mais de um processo. Este compartilhamento entre processos pode ser complexo e pouco eficiente.
- No caso do uso de threads, já existe um espaço de memória compartilhado entre as threads no mesmo processo que as criou baixa complexidade. Por isso, o uso de threads é frequente no paradigma de memória compartilhada.

PROGRAMAÇÃO COM MEMÓRIA COMPARTILHADA



OpenMP

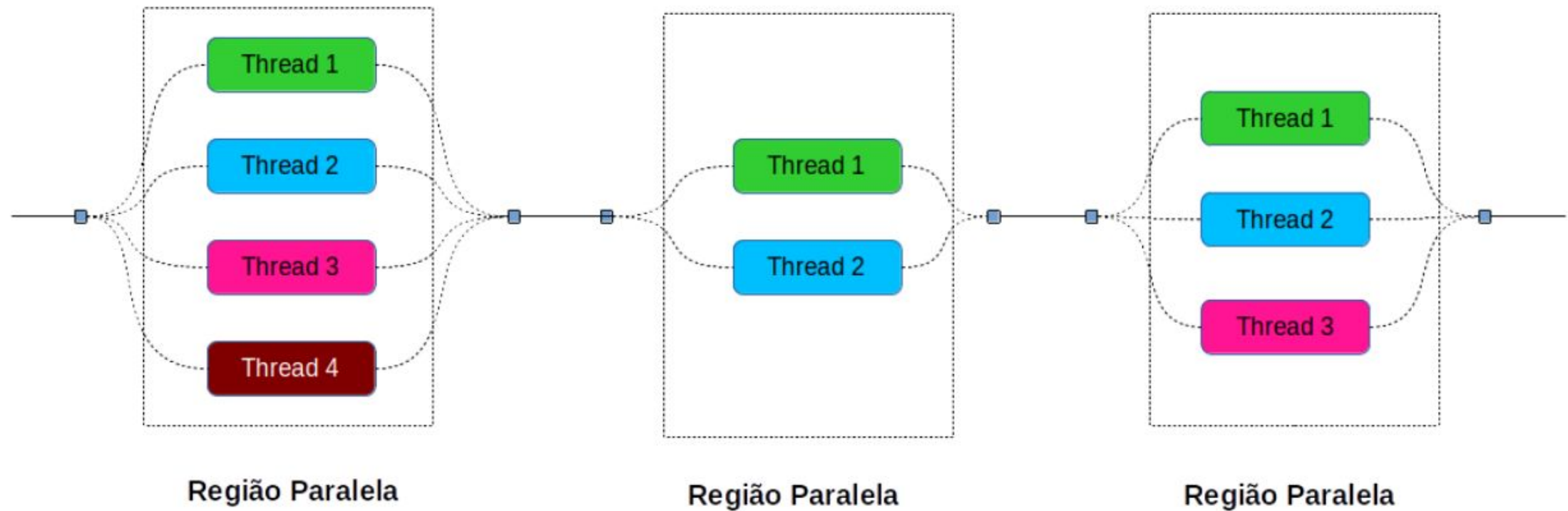
- Uma API para programação paralela em memória compartilhada.
- Cada thread ou processo pode potencialmente ter acesso a toda a memória disponível.
- **O sistema é visto** como uma coleção de núcleos ou CPUs, todos com acesso à memória principal.
 - A execução será em **paralelo se houver um número suficiente** de processadores físicos. (Ex.: 16 threads em uma máquina com 8 núcleos)

OpenMP

- Uma API para programação paralela em memória compartilhada.
- Cada thread ou processo pode potencialmente ter acesso a toda a memória disponível.
- **O sistema é visto** como uma coleção de núcleos ou CPUs, todos com acesso à memória principal.
 - A execução será em **paralelo se houver um número suficiente** de processadores físicos. (Ex.: 16 threads em uma máquina com 8 núcleos)
 - Troca de contexto.
 - Apenas 8 executam ao mesmo tempo
 - As outras 8 ficam esperando e são alternadas

OpenMP

Join/Fork



Algumas nomenclaturas

- **Pragmas**

- Instruções normalmente adicionadas a um sistema para permitir comportamentos que não fazem parte da especificação básica de C.
- Compiladores que não suportam as pragmas as ignoram.

- **Diretivas**

- Linha especial de código-fonte com significado especial apenas para determinados compiladores

- **Clause**

- Texto que modifica a diretiva

```
#include <stdio.h>
#include <omp.h>
int main ( int argc , char * argv []) {
    /* omp_parallel .c */
    #pragma omp parallel num_threads (4)
    {
        int tid = omp_get_thread_num() ;
        printf (" Alo da thread %d. \n", tid );
    }
    return 0;
}
```

PROGRAMAS MULTITAREFAS

Tarefa 5:

Implemente um programa em C **que conte quantos números primos existem entre 2 e um valor máximo n .**

Depois, paralelize o laço principal usando a diretiva

#pragma omp parallel for
sem alterar a lógica original. compare o tempo de execução e os resultados das versões sequencial e paralela. observe possíveis diferenças no resultado e no desempenho, e reflita sobre os desafios iniciais da programação paralela, como correção e distribuição de carga.

- Por que é mais difícil programar em paralelo?
 - Sincronização, comunicação e equilíbrio de carga

```
$ gcc -fopenmp meuprog.c -o meuprog
```

```
$ export OMP_NUM_THREADS=2; time meupro
```

Referências

